

[Directory](#) > [R2](#) >[R2 Data Catalog](#) >[Connect to Iceberg engines](#) >[Spark \(Scala\)](#)

Copy page



Spark (Scala)

Below is an example of how you can build an [Apache Spark ↗](#) application (with Scala) which connects to R2 Data Catalog. This application is built to run locally, but it can be adapted to run on a cluster.

Prerequisites

- Sign up for a [Cloudflare account ↗](#).
- [Create an R2 bucket](#) and [enable the data catalog](#).
- [Create an R2 API token](#) with both [R2 and data catalog permissions](#).

- Install Java 17, Spark 3.5.3, and SBT 1.10.11
 - Note: The specific versions of tools are critical for getting things to work in this example.
 - Tip: ["SDKMAN"](#)  is a convenient package manager for installing SDKs.

Example usage

To start, create a new empty project directory somewhere on your machine.

Inside that directory, create the following file at `src/main/scala/com/example/R2DataCatalogDemo.scala`. This will serve as the main entry point for your Spark application.

```
package com.example

import org.apache.spark.sql.SparkSession

object R2DataCatalogDemo {
  def main(args: Array[String]): Unit = {

    val uri = sys.env("CATALOG_URI")
    val warehouse = sys.env("WAREHOUSE")
    val token = sys.env("TOKEN")

    val spark = SparkSession.builder()
      .appName("My R2 Data Catalog Demo")
      .master("local[*]")
      .config("spark.sql.extensions",
"org.apache.iceberg.spark.extensions.IcebergSpark")
      .config("spark.sql.catalog.mydemo",
"org.apache.iceberg.spark.SparkCatalog")
      .config("spark.sql.catalog.mydemo.type", "r2")
      .config("spark.sql.catalog.mydemo.uri", uri)
      .config("spark.sql.catalog.mydemo.warehouse", warehouse)
      .config("spark.sql.catalog.mydemo.token", token)
      .getOrCreate()
```

```
import spark.implicit._

val data = Seq(
  (1, "Alice", 25),
  (2, "Bob", 30),
  (3, "Charlie", 35),
  (4, "Diana", 40)
).toDF("id", "name", "age")

spark.sql("USE mydemo")

spark.sql("CREATE NAMESPACE IF NOT EXISTS demoNamespace")

data.writeTo("demoNamespace.demotable").createTableIfNotExists()

val readResult = spark.sql("SELECT * FROM
demoNamespace.demotable WHERE age > 30")
println("Records with age > 30:")
readResult.show()
}
}
```

For building this application and managing dependencies, we will use [sbt \("simple build tool"\)](#).

The following is an example `build.sbt` file to place at the root of your project. It is configured to produce a "fat JAR", bundling all required dependencies.

```
name := "R2DataCatalogDemo"
```

```
version := "1.0"
```

```
val sparkVersion = "3.5.3"
```

```
val icebergVersion = "1.8.1"
```

// You need to use binaries of Spark compiled with either 2.12 or 2.13; and 2.12 is more common.

// If you download Spark 3.5.3 with sdkman, then it comes with 2.12.18

```
scalaVersion := "2.12.18"
```

```
libraryDependencies += Seq(
```

```
    "org.apache.spark" %% "spark-core" %  
    sparkVersion,
```

```
    "org.apache.spark" %% "spark-sql" %  
    sparkVersion,
```

```
    "org.apache.iceberg" % "iceberg-core" %  
    icebergVersion,
```

```
    "org.apache.iceberg" % "iceberg-spark-  
runtime-3.5_2.12" % icebergVersion,
```

```
    "org.apache.iceberg" % "iceberg-aws-  
bundle" % icebergVersion,  
)  
  
// build a fat JAR with all dependencies  
assembly / assemblyMergeStrategy := {  
    case PathList("META-INF", "services", xs @  
_*) => MergeStrategy.concat  
    case PathList("META-INF", xs @ _*) =>  
MergeStrategy.discard  
    case "reference.conf" =>  
MergeStrategy.concat  
    case "application.conf" =>  
MergeStrategy.concat  
    case x if x.endsWith(".properties") =>  
MergeStrategy.first  
    case x => MergeStrategy.first  
}  
  
// For Java 17 Compatability  
Compile / javacOptions ++= Seq("--release",  
"17")
```

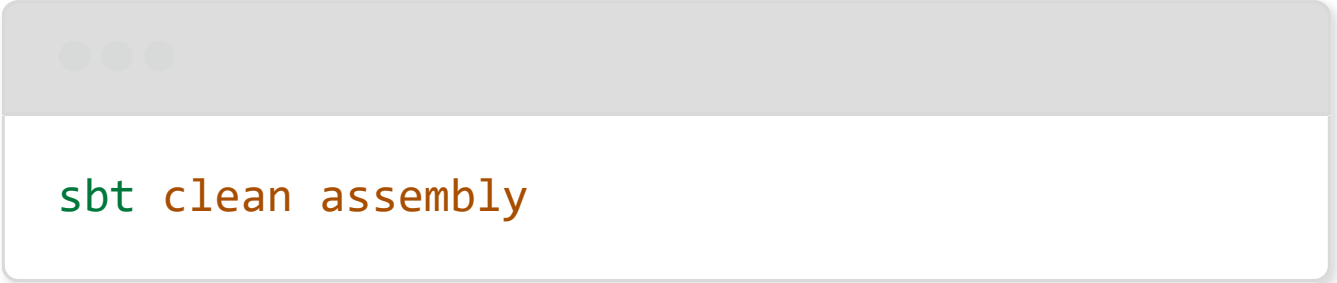
To enable the [sbt-assembly-plugin ↗](#) (used to build fat JARs), add the following to a new file at `project/assembly.sbt`:

```
addSbtPlugin("com.eed3si9n" % "sbt-assembly" % "1.2.0")
```

Make sure Java, Spark, and sbt are installed and available in your shell. If you are using SDKMAN, you can install them as shown below:

```
sdk install java 17.0.14-amzn
sdk install spark 3.5.3
sdk install sbt 1.10.11
```

With everything installed, you can now build the project using sbt. This will generate a single bundled JAR file.

A terminal window with a light gray title bar containing three window control buttons. The main area is white and displays the command 'sbt clean assembly' in a monospaced font. 'sbt' is green, 'clean' is orange, and 'assembly' is orange.

```
sbt clean assembly
```

After building, the output JAR should be located at `target/scala-2.12/R2DataCatalogDemo-assembly-1.0.jar`.

To run the application, you will use `spark-submit`. Below is an example shell script (`submit.sh`) that includes the necessary Java compatibility flags for Spark on Java 17:

```
# We need to set these "--add-opens" so that Spark
can run on Java 17 (it needs access to
# parts of the JVM which have been modularized as
internal).
JAVA_17_COMPATABILITY="--add-
opens=java.base/sun.nio.ch=ALL-UNNAMED --add-
opens=java.base/java.nio=ALL-UNNAMED --add-
opens=java.base/java.lang=ALL-UNNAMED --add-
opens=java.base/java.util=ALL-UNNAMED --add-
opens=java.base/java.util.concurrent=ALL-UNNAMED"

spark-submit \
--conf
"spark.driver.extraJavaOptions=$JAVA_17_COMPATABILITY"
--conf
"spark.executor.extraJavaOptions=$JAVA_17_COMPATABILITY" \
--class com.example.R2DataCatalogDemo target/scala-
2.12/R2DataCatalogDemo-assembly-1.0.jar
```

Before running it, make sure the script is executable:

```
chmod +x submit.sh
```

At this point, your project directory should be structured like this:

```
graph TD
    Root[ ] --- Makefile[Makefile]
    Root --- README[README.md]
    Root --- build_sbt[build.sbt]
    Root --- project[project]
    Root --- spark_submit[spark-submit.sh]
    Root --- src[src]
    project --- assembly_sbt[assembly.sbt]
    project --- build_properties[build.properties]
    project --- project_files[project]
    src --- main[main]
    main --- scala[scala]
    scala --- com[com]
    com --- example[example]
    example --- R2DataCatalogDemo[R2DataCatalogDemo.scala]
```

Before submitting the job, make sure you have the required environment variable set for your catalog URI, warehouse, and [Cloudflare API token](#).

```
export CATALOG_URI=  
export WAREHOUSE=  
export TOKEN=
```

You are now ready to run the job:

```
./submit.sh
```

Was this helpful?



Previous

← Spark (PySpark)

Next

StarRocks →

Last updated: Jul 8, 2025

Resources

[API](#)

[New to Cloudflare?](#)

[Directory](#)

[Sponsorships](#)

[Open Source](#)

Company

[cloudflare.com](#)

[Our team](#)

[Careers](#)

Support

[Help Center](#)

[System Status](#)

[Compliance](#)

[GDPR](#)

Tools

[Cloudflare Radar](#)

[Speed Test](#)

[Is BGP Safe Yet?](#)

[RPKI Toolkit](#)

[Certificate Transparency](#)

Community

 [X](#)

 [Discord](#)

 [YouTube](#)



© 2025 Cloudflare, Inc. • [Privacy Policy](#) • [Terms of Use](#)

• [Report Security Issues](#) • [Trademark](#) •  [Cookie Preferences](#)